VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING



ESSAY REPORT

SUBJECT: DIGITAL IMAGE PROCESSING

Subject Code: 402084

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING



ESSAY REPORT

SUBJECT: DIGITAL IMAGE PROCESSING

PROJECT : Image processing from video to detect offside situations in football

Name of student	StudentID
Đỗ Hoàng Minh Hân	423H0029
Nguyễn Minh Hoàng	423H0093
Doãn Tùng Lâm	420H0205

Dr. Le Anh Vu

HO CHI MINH CITY, 2025

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Dr. Le Anh Vu for his devoted guidance and valuable knowledge throughout the digital image processing course. His engaging lectures and practical insights have greatly enhanced my understanding of industrial automation systems and inspired me to complete this report with enthusiasm and confidence.

Despite our best efforts, there may still be some limitations in the content and data presented. I sincerely hope to receive your understanding and constructive feedback to help us further improve.

Wishing you good health and continued success in your teaching career, and may you always keep your passion for inspiring students in the field of automation.

Ho Chi Minh City, 20th March 2026

Author

Do Hoang Minh Han

BẢNG PHÂN CÔNG NHIỆM VỤ

Member	Student ID	Task assignment
Đỗ Hoàng Minh Hân (Leader)	423H0029	Coding and creating user interfaces in Tia Portal, making slides and PowerPoint presentations, and writing reports/essays.
Nguyễn Minh Hoàng	423H0093	Coding and creating user interfaces in Tia Portal, and PowerPoint presentations, and writing reports/essays.
Doãn Tùng Lâm	420H0205	Create a Tia Portal interface to support essay/report creation.

CONTENT

LIST OF FIGURES ..... **VII**

CHAPTER 1: INTRODUCTION **1**

1.1 Reason for Choosing the Topic 1

1.2 Objectives of the Study 1

1.3 Research Methodology 2

1.4 Research Object and Scope 2

1.4.1 Research object: *2*

1.4.2 Research scope: *3*

1.5 Scientific and Practical Significance 3

1.5.1 Scientific significance: *3*

1.5.2 Practical significance: *3*

CHAPTER 2: THEORETICAL BASIS OF OFFSIDE DETECTION IN SOCCER USING IMAGE PROCESSING **5**

2.1 Overview of Image Processing 5

2.1.1 Image Acquisition *5*

2.1.2 Image Preprocessing *6*

2.1.3 Image Segmentation *6*

2.1.4 Feature Representation *6*

2.1.5 Object Recognition and Interpretation *6*

2.1.6 Knowledge-Based System *6*

2.1.6 Knowledge-Based System *6*

2.2 Getting the Abscissa Coordinate 7

2.2.2 Geometric Relationship Between Image and Real Coordinates *8*

2.2.3 Camera Perspective and Optical Relationship *9*

2.2.4 Determining the Real Position of the Object *10*

2.3 Getting Breadth Coordinate 10

2.3.1 Geometric Construction of the Trapezoid Model *11*
2.3.2 Geometric Relationships in the Trapezoid *11*
2.3.3 Optical Relationship Between Real Object and Image *12*
2.3.4 Determination of the Breadth Coordinate *13*
2.4 Calculate Coordinates of Objects 14
2.4.1 Determination of Intermediate Parameter *14*
2.4.2 Transformation of the Abscissa Coordinate *15*
2.4.3 Determination of the Breadth Coordinate *16*
2.5. Detection of Lines on the Soccer Pitch 17
2.5.1. Field Characteristics and Color Space Selection *17*
2.5.2. Edge Detection on the Soccer Field *18*
2.5.3. Line Detection and Filtering using Hough Transform *18*
2.6. Vanishing Point Estimation 19
2.6.1. Perspective Geometry and Vanishing Point *19*
2.6.2. Vanishing Point Estimation using Line Intersections and RANSAC *20*
2.6.3. Final Vanishing Point Determination and Application *20*
2.7. Keypoint Estimation and Offside Line Construction 21
2.7.1. Importance of Keypoint Estimation *21*
2.7.2. Virtual Line Generation from Vanishing Point *22*
2.7.3. Offside Line Selection *22*
CHAPTER 3: IMPLEMENTATION AND SYSTEM DESIGN 23
3.1 Development Environment and Tools 23
3.2 Software Libraries and Tools 24
3.2.1 Player Detection Using YOLO 24
3.2.2 Team Classification Using K-Means *24*
3.2.3 Offside Line Estimation Using Kalman Filter *25*
3.2.4 OpenCV *26*
3.2.5 DeepSORT for Player Tracking ..... ***Error! Bookmark not defined.***
3.3 Flowchart of the offside detection algorithm. 3.4. System Workflow of the Offside Detection System 28
3.4.1. Data Acquisition and Pre-processing *29*

3.4.2. Player Detection and Tracking.....*29*
3.4.3. Team Classification and Attack Direction Estimation *30*

LIST OF FIGURES

Chapter 1

Figure 1.1: Requriment of Topic ix

Chapter 2

Figure 2.1:Basic steps in image processing 5

Figure 2.2: Imagehalf playground from camera behind the breadth line 7

Figure 2.3: Trapezoid ABCD as half-field view from abscissa-line camera ...8

Figure 2.4: Image half playground from camera behind the abscissa line 10

Figure 2.5: Getting Breadth Coordinate.....10

Figure 2.6: Trapezoidal Half-Field View from Goal-Line Camera 11

Figure 2.7: Distance Measurement from Breadth-Line Camera.....14

Figure 2.8: Saturation Channel of HSV space 17

Figure 2.9: Edge detection result using Canny detector. 18

Figure 2.10: Final detected lines on the soccer pitch. 19

Figure 2.11: Vanishing point when the camera is centered on the pitch.....19

Figure 2.12: Vanishing point when the camera is viewing one end of the pitch……21

Figure 2.13: Detected keypoints on players.....21

Figure 2.14: Lines drawn from all detected keypoints 22

Figure 2.15: Final offside line corresponding to the last defender 22

Chapter 3

Figure 3.1:Development environment using PyCharm IDE.....23

Figure 3.2: Player detection using YOLOv8. 24

Figure 3.3: Team classification using K-Means clustering. 25

Figure 3.4: Smoothed offside line estimation using Kalman Filter 25

Figure 3.5: OpenCV image processing library 26

Chapter 4

Figure 4.1: Communication network configuration**Error! Bookmark not defined.**

Figure 4.2: Simulate System in WinCC Tiaportal**Error! Bookmark not defined.**

Figure 4.3: Control Panel Box .....**Error! Bookmark not defined.**

Figure 4.4: Automatic Control Panel .....**Error! Bookmark not defined.**

Figure 4.5: Manual Control Panel.....**Error! Bookmark not defined.**

TOPIC REQUIREMENTS

Requirements of Project

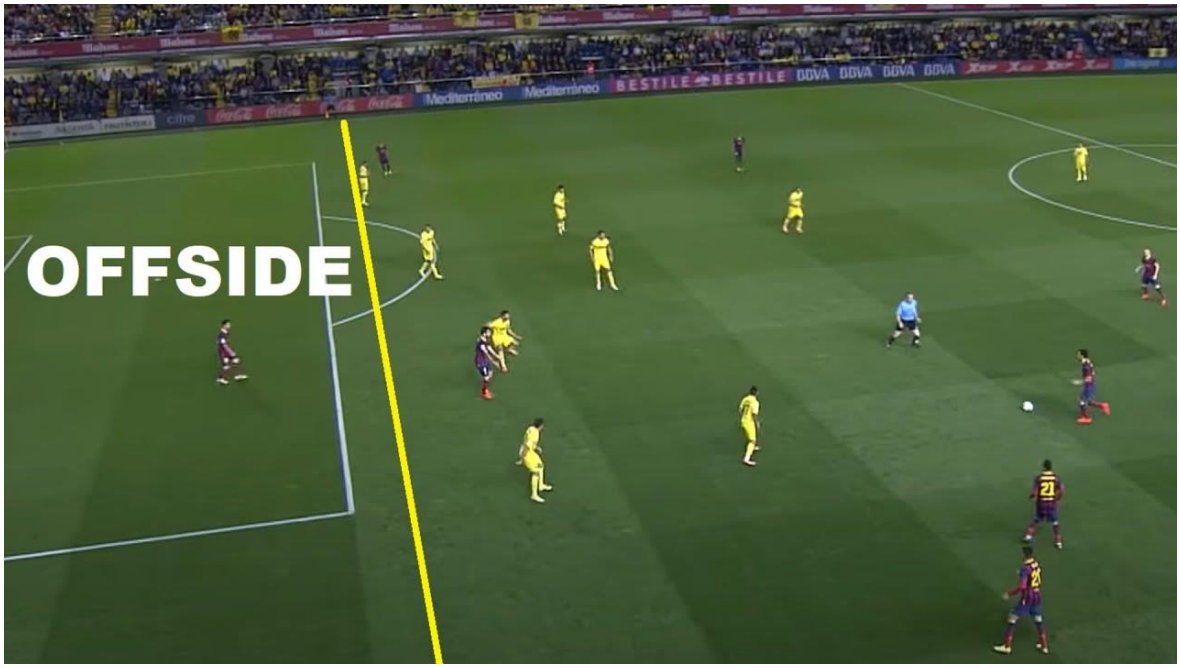


Figure 1.1: Requriment of Topic

CHAPTER 1: INTRODUCTION

1.1 Reason for Choosing the Topic

Football is one of the most popular sports in the world, attracting millions of fans and viewers. However, during matches, referees sometimes face difficulties in making accurate decisions, especially in offside situations. Offside decisions often happen very quickly and depend on the relative positions of players at a specific moment, which can be difficult to judge with the human eye alone.

With the rapid development of computer vision and digital image processing technologies, it has become possible to analyze images and videos automatically to support decision-making in sports. Image processing techniques can help detect players, determine their positions on the field, and analyze whether an offside situation occurs.

The reason for choosing this topic is that it combines theoretical knowledge of digital image processing with a practical and interesting application in sports analysis. By processing frames extracted from football match videos, the system can assist in identifying potential offside situations more objectively.

In addition, this topic provides an opportunity to explore several important techniques in digital image processing such as object detection, line detection, and video frame analysis. Therefore, studying and implementing an offside detection system from video is both academically meaningful and practically useful.

1.2 Objectives of the Study

The main objective of this study is to apply digital image processing techniques to analyze football match videos to detect potential offside situations. By processing frames extracted from video footage, the system aims to identify players, determine their positions on the field, and assist in recognizing offside conditions.

Specifically, the objectives of this study include:

- To study the fundamental concepts and techniques of digital image processing used in video analysis.

- To extract frames from football match videos for image processing and analysis.
- To apply image processing methods such as object detection, edge detection, and line detection to identify players and important field lines.
- To analyze the relative positions of attacking players and defending players to determine potential offside situations.
- To develop a simple system or algorithm that demonstrates how digital image processing can support offside detection in football matches.

Through this study, the project aims to demonstrate how digital image processing can be applied in real-world scenarios, particularly in sports analysis, while also helping students better understand and practice the techniques learned in the Digital Image Processing course.

1.3 Research Methodology

In this study, the research methodology is based on applying digital image processing techniques to analyze football match videos for offside detection. The research process includes several main steps.

First, relevant theories and previous studies related to digital image processing and computer vision are reviewed. This helps to understand the fundamental techniques used in image analysis, such as object detection, edge detection, and video frame processing. Second, football match videos are collected as input data for the system.

The videos are then processed by extracting individual frames, which allows image processing algorithms to analyze each frame separately. Third, image processing techniques are applied to detect important elements in the frame. These techniques include detecting players, identifying field lines, and analyzing the positions of players on the field. Edge detection and line detection methods can be used to determine reference lines on the field, which are important for analyzing offside situations. Finally, the positions of attacking players and defending players are compared to determine whether an offside situation may occur. The results are then analyzed to evaluate how effectively digital image processing techniques can assist in detecting offside situations from video data.

1.4 Research Object and Scope

1.4.1 Research object:

The research object of this study is football match video data that is used to analyze and detect potential offside situations. The system focuses on processing images extracted from video frames to identify important elements such as players and field

lines. Through digital image processing techniques, the study analyzes the positions of attacking players and defending players in each frame of the video. These elements are essential for determining whether an offside situation occurs during a football match.

1.4.2 Research scope:

The scope of this research focuses on applying digital image processing techniques to detect potential offside situations from football match videos. The system processes video frames to identify players and analyze their positions relative to the defensive line. This study mainly focuses on basic techniques in digital image processing and computer vision, such as frame extraction, object detection, edge detection, and line detection.

The system is designed as a simplified model for educational purposes and may not achieve the same level of accuracy as professional systems used in real football competitions. In addition, the research is limited to analyzing 2D video footage and depends on the quality and camera angle of the input video. Therefore, some complex match situations or unclear frames may affect the accuracy of the offside detection results.

1.5 Scientific and Practical Significance

1.5.1 Scientific significance:

This research contributes to the study and application of digital image processing techniques in the field of sports video analysis. By applying image processing methods such as frame extraction, object detection, edge detection, and line detection, the study demonstrates how theoretical knowledge in digital image processing can be used to analyze real-world visual data.

The research also helps to illustrate how computer vision techniques can be used to detect and analyze player positions in football match videos. Through this process, the study provides a basic model for understanding how automated systems can assist in identifying offside situations. This contributes to expanding the application of digital image processing in sports technology and intelligent video analysis.

1.5.2 Practical significance:

In practical terms, this research demonstrates the potential use of digital image processing to support decision-making in football matches. Offside situations are often difficult to determine accurately in real time because they occur very quickly and depend on the exact positions of players. By analyzing video frames and detecting

player positions automatically, the proposed approach can help assist referees or analysts in identifying potential offside situations more objectively.

Although the system developed in this study is mainly for educational and research purposes, it shows how image processing technology can be applied to improve fairness and accuracy in sports analysis. Furthermore, the project provides a simple demonstration of how video-based analysis can be developed using digital image processing techniques, which can be expanded or improved in future research.

CHAPTER 2: THEORETICAL BASIS OF OFFSIDE DETECTION IN SOCCER USING IMAGE PROCESSING

2.1 Overview of Image Processing

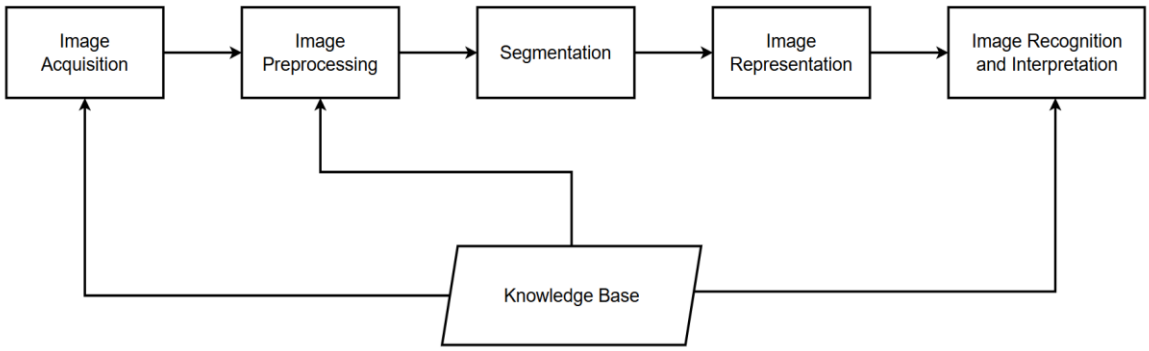


Figure 2.1:Basic steps in image processing

Image processing is an important field in computer vision that focuses on analyzing and manipulating digital images to extract useful information. In a typical system, an image is taken as input, processed through several steps, and transformed into a more meaningful output for visualization or analysis. A digital image is represented as a two-dimensional matrix of pixels, where each pixel contains intensity or color information. Based on this representation, various algorithms can be applied to analyze image content and structure.

A standard image processing framework includes the following main stages:

- **Image acquisition:** capturing images from cameras or sensors
- **Preprocessing:** improving image quality by reducing noise and enhancing contrast
- **Analysis and segmentation:** extracting important features and separating objects
- **Recognition and interpretation:** identifying objects and assigning meaning

The main goal of image processing is to automatically extract useful information from visual data. It is widely applied in areas such as medical imaging, security systems, and industrial inspection.

2.1.1 Image Acquisition

Image acquisition is the first step, where images are captured and converted into digital form. In video-based systems, this also involves extracting frames from video

sequences. In this study, football match videos are used as input, and individual frames are analyzed to detect player positions and potential offside situations.

2.1.2 Image Preprocessing

Preprocessing aims to improve image quality before further analysis. Common techniques include noise reduction, contrast enhancement, and brightness adjustment. This step helps improve the accuracy of later processes such as object detection and segmentation.

2.1.3 Image Segmentation

Segmentation divides an image into meaningful regions to separate objects from the background. This process is typically based on similarities in pixel properties such as color or intensity, allowing the system to focus on relevant areas and improve efficiency.

2.1.4 Feature Representation

After segmentation, image regions are represented using descriptive features instead of raw pixels. These features (such as shape and size) provide a compact and meaningful way to describe objects and are essential for recognition tasks.

2.1.5 Object Recognition and Interpretation

Object recognition identifies and classifies objects based on extracted features. Image interpretation extends this process by assigning meaning to objects within a specific context, supporting decision-making in the system.

2.1.6 Knowledge-Based System

A knowledge-based system supports processing by providing reference data, rules, or trained models. This helps improve accuracy and reliability, especially in complex or noisy environments.

2.1.6 Knowledge-Based System

To reduce data redundancy, images are often represented in more compact forms. Common techniques include:

- **Run-Length Encoding (RLE):** compresses consecutive pixels
- **Chain Code:** represents object boundaries
- **Quadtree:** divides images into hierarchical regions

These methods improve storage efficiency and support faster processing.

2.2 Getting the Abscissa Coordinate



Figure 2.2: Imagehalf playground from camera behind the breadth line

In a soccer video analysis system, the positions of players and the ball are initially obtained from the captured image frames. These positions are represented in the image coordinate system. However, the image coordinates do not directly correspond to the real positions of objects on the football field. Therefore, it is necessary to calculate the abscissa coordinate (X coordinate) of each detected object. During a soccer broadcast, an imaginary line parallel to the goal line is often used as the offside reference line.

The intersection between this line and the ground line of the field determines the horizontal position of the player or the ball. Even a small error in determining this coordinate may lead to incorrect analysis in offside detection. According to the geometric perspective of the camera, each half of the football field appears as a trapezoidal shape in the captured image. By extending the sides of this trapezoid until they intersect at a distant point, a perspective model of the field can be established. A line is then drawn from this intersection point through the detected object. The point where this line intersects the ground line of the field represents the real X coordinate of the object.

2.2.1 Geometric Model of the Playground

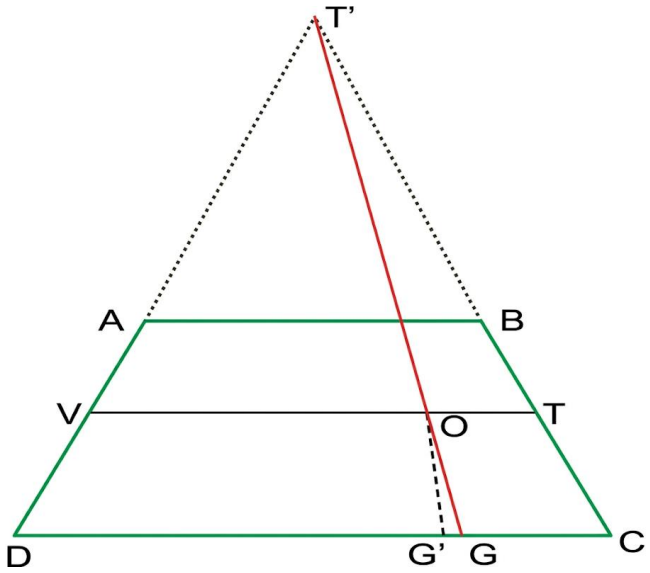


Figure 2.3: Trapezoid ABCD as half-field view from abscissa-line camera

From the camera view, the half football field can be approximated as a trapezoid labeled ABCD. Suppose O represents the position of an object (such as a player) in the image. From point **O**, a line is drawn parallel to the two parallel sides of the trapezoid until it intersects the edges of the trapezoid at points V and T. The segment VO represents the distance from the object to the midfield line in the image plane. However, this distance corresponds only to a virtual distance in the image. To obtain the real distance on the football field, it must be converted using geometric relationships derived from the trapezoidal model. Let DC represent the real width of the field at the bottom boundary. The intersection of the extended trapezoid sides defines a remote perspective point. A line drawn from this point through O intersects the ground line DC at point G, which represents the real horizontal location of the object.

2.2.2 Geometric Relationship Between Image and Real Coordinates

Based on the similarity of triangles formed in the geometric configuration, proportional relationships between the image distance and the real distance can be derived. Considering triangles LCD and IDG, where VT is parallel to DC, the following relationship can be obtained:

$$\frac{VO}{DG} = \frac{IV}{ID} \times \frac{VT}{DC}$$

Since the ratios of similar triangles are equal, the relationship can be simplified as:

$$\frac{VO}{DG} = \frac{VT}{DC}$$

where:

- VO: the virtual distance of the object measured in the image
- DG: the real distance of the object on the football field
- VT: the distance between reference points in the image plan
- DC: the corresponding real distance on the field
- IV, ID: reference distances derived from the geometric perspective model

2.2.3 Camera Perspective and Optical Relationship

To estimate the real position of an object on the football field, the relationship between the camera imaging system and the geometric structure of the playground must be considered. According to the optical imaging principle, the following equation can be expressed:

$$\frac{1}{F} = \frac{1}{p} + \frac{1}{q}$$

Let:

- F be the focal length of the camera lens
- p be the distance from the camera to the object
- q be the distance from the camera to the image plane

The ratio between the image size and the real object size is also proportional to the ratio between **q** and **p**, which can be written as:

$$\frac{q}{p} = \frac{\text{Size of Picture}}{\text{Size of Object}}$$

Where:

- **Size of Picture**: size of the object measured in the image
- **Size of Object**: actual size of the object in the real world

From this relationship, the image distance q can be expressed as:

$$q = \frac{\text{Size of Picture} \times p}{\text{Size of Object}}$$

These optical relationships help explain how the camera transforms real-world objects into image projections.

2.2.4 Determining the Real Position of the Object

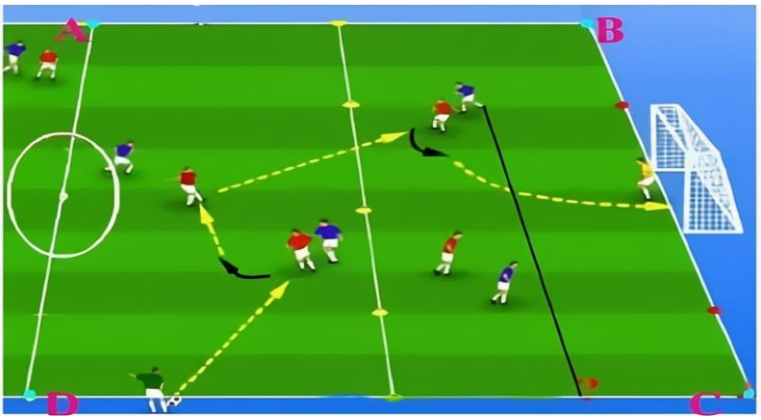


Figure 2.4: Image half playground from camera behind the abscissa line

Finally, the real position of the object on the football field can be determined using the geometric and optical relationships described above. The line passing through the object O intersects the ground line at G, which coincides with G′**.** Therefore, the distance DG represents the real X-coordinate (abscissa coordinate) of the detected object on the field. This coordinate can then be used to compare the positions of attacking players and defenders to determine possible offside situations.

2.3 Getting Breadth Coordinate

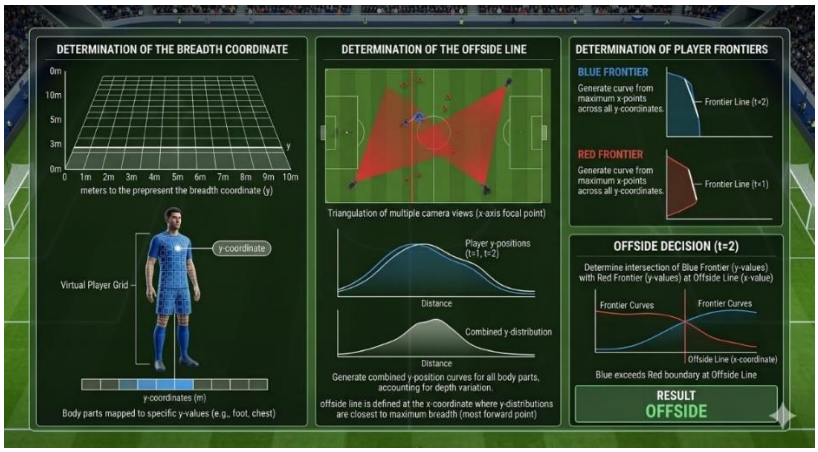


Figure 2.5: Getting Breadth Coordinate

After determining the abscissa coordinate of the detected object, the next step is to determine the breadth coordinate **(**Y-coordinate**)**, which represents the depth position of the player or the ball relative to the camera. Because the football field appears as a trapezoidal shape in the captured image, the breadth coordinate cannot be obtained

directly from the image. Instead, geometric relationships within the trapezoidal model must be used to estimate the real depth location.

2.3.1 Geometric Construction of the Trapezoid Model

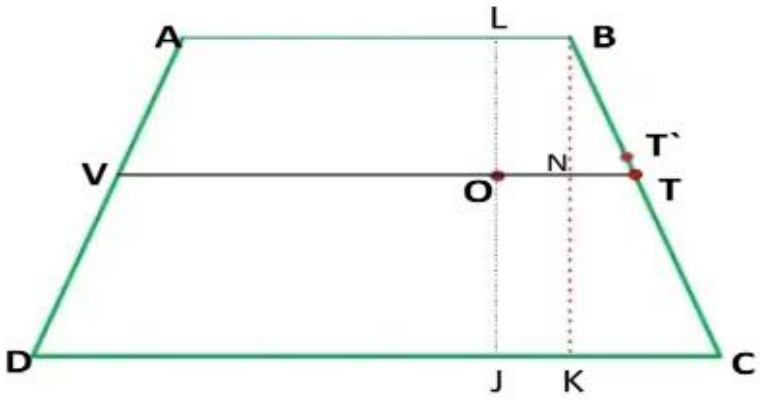


Figure 2.6: Trapezoidal Half-Field View from Goal-Line Camera

When the football field is viewed from a camera placed behind the goal line, half of the playground can be approximated as a trapezoid **ABCD**. In this trapezoidal model:

- **AB** represents the upper boundary of the field in the image,
- **DC** represents the lower boundary,
- **AD** and **BC** correspond to the two lateral sides of the field.

Let **O** denote the position of the detected object in the image. To determine the depth coordinate, a line is drawn through point **O** parallel to the baseline **DC**. This line intersects the sides of the trapezoid at two points denoted as **V** and **T**. From point **T**, a perpendicular line is extended upward and downward so that it intersects the upper edge **AB** at point **L** and the lower edge **DC** at point **J**. Afterward, another segment **BK** is constructed such that it is parallel to **JL**. This geometric configuration provides the basis for establishing proportional relationships between image distances and real-world distances.

2.3.2 Geometric Relationships in the Trapezoid

Based on the trapezoidal structure and the parallel lines constructed in the previous step, several relationships between the line segments can be obtained.

+ First, because BK is drawn parallel to JL, the following equality holds:

$$JL = BK$$

+ Additionally, the horizontal distance between the object and the vertical projection can be expressed as:

$$NK = OJ$$

+ Using the similarity between the triangles formed by these lines, the following proportional relationship can be derived:

$$\frac{NK}{BK} = \frac{BC}{CT}$$

+ From this proportional equation, the virtual image distance **CT** can be obtained as:

$$CT = \frac{BC \cdot NK}{BK}$$

where:

- NK: represents the distance measured in the image,
- BK: corresponds to the real breadth distance,
- BC: represents the width of the trapezoidal model,
- CT: denotes the virtual distance associated with the projected image.

This relationship allows the conversion of distances measured in the image plane into values related to the actual geometry of the football field.

2.3.3 Optical Relationship Between Real Object and Image

The captured image is produced by a camera lens, which follows the fundamental principle of optical projection. Therefore, the relationship between the real object and the image formed by the camera can be described using the thin lens equation.

Let:

- F be the focal length of the camera lens,
- p be the distance between the camera and the real object,
- q be the distance between the camera and the image plane.

$$\frac{1}{F} = \frac{1}{p} + \frac{1}{q}$$

Furthermore, the ratio between the size of the virtual image and the size of the real object can be written as:

$$\frac{q}{p} = \frac{\text{Size of Virtual Picture}}{\text{Size of Real Object}}$$

In the trapezoidal geometric model used in the soccer field, this relationship can also be expressed using the previously obtained geometric measurements. Specifically, the ratio between the image distance and the real object distance is equivalent to the ratio between the measured segment **NK** and the virtual segment **CT**:

$$\frac{q}{p} = \frac{NK}{CT}$$

From this equation, the distance **q** can be expressed as:

$$q = p \cdot \frac{NK}{CT}$$

2.3.4 Determination of the Breadth Coordinate

Once the geometric and optical relationships have been established, the real breadth coordinate of the detected object can be determined. The value obtained from the trapezoidal model corresponds to the depth position of the player relative to the camera. By combining the proportional relationships of the trapezoid and the optical projection equations, the system converts the measured image distances into the real-world breadth coordinate of the object. This value represents the distance of the player from the baseline of the football field. The calculated breadth coordinate will later be combined with the abscissa coordinate obtained in Section 2.2 to determine the complete real-world position of the object on the field.

2.4 Calculate Coordinates of Objects

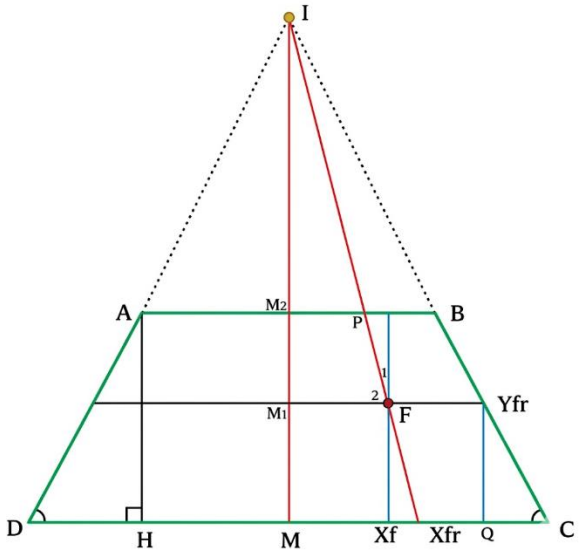


Figure 2.7: Distance Measurement from Breadth-Line Camera

After obtaining the geometric relationships from the previous sections, the next step is to determine the real-world coordinates of the detected objects, including players and the ball. The main objective of this stage is to convert the coordinates from the image plane into the actual positions on the football field.

In this process, two coordinates are considered:

- The abscissa coordinate (X), which represents the horizontal position of the object
- The breadth coordinate (Y)**,** which represents the depth position relative to the camera

Let X_f and Y_f denote the coordinates of the object in the image, while X_{fr} and Y_{fr} represent the corresponding real-world coordinates.

2.4.1 Determination of Intermediate Parameter

To establish the relationship between the image and real-world coordinates, an intermediate value is introduced based on the trapezoidal model of the football field. From the geometric configuration, the following trigonometric relationship is obtained:

$$\tan(\angle ADH) = \frac{AH}{DH} = \frac{YA}{XA}$$

Using this relation, the intermediate distance IM can be computed as:

$$IM = DM \cdot \tan(\angle ADH)$$

Substituting the expression of the tangent:

$$IM = DM \cdot \frac{YA}{XA}$$

Since $\mathrm{DM} = \frac{\mathrm{DC}}{2}$, the equation becomes: $\mathrm{IM} = \left(\frac{\mathrm{DC}}{2}\right) \times \left(\frac{\mathrm{YA}}{\mathrm{XA}}\right)$

This value serves as a reference distance for mapping the object position from the image to the field.

2.4.2 Transformation of the Abscissa Coordinate

To compute the real horizontal position of the object, the perspective relationship between the image coordinate and the field coordinate is considered.

From the projection geometry, we have:

$$\tan(\angle F2) = \frac{IM1}{M1F}$$

where: $\mathrm{IM1} = \mathrm{IM} - \mathrm{Yf}$ and $\mathrm{M1F} = Xf - DM$

Rewriting the relationship using cotangent:

$$\cot(\angle F2) = \frac{Xf - DM}{IM - Yf}$$

Because of the similarity between triangles in the projection:

$$M_{Xfr} = IM \cdot \cot(\angle F2)$$

Thus, the horizontal displacement in the real coordinate system is:

$$M_{Xfr} = IM \cdot \frac{(Xf - DM)}{(IM - Yf)}$$

The real abscissa coordinate is then given by:

$$Xfr = \frac{DC}{2} + M_{Xfr}$$

Substituting the expression of $\mathrm{M_{Xfr}}$

$$Xfr = \frac{DC}{2} + IM \cdot \frac{(Xf - DM)}{(IM - Yf)}$$

By replacing IM, we obtain:

$$Xfr = \frac{DC}{2} + \frac{\left(\frac{DC}{2} \cdot \frac{YA}{XA}\right)\left(Xf - \frac{DC}{2}\right)}{\left(\frac{DC}{2} \cdot \frac{YA}{XA}\right) - Yf}$$

Finally, the real horizontal position of the player is:

$$X_{soccerPlayer} = Xfr + DC$$

Adjustment for Field Representation

To account for the full field representation, the coordinate can be further adjusted based on the field dimensions. The extended expression of the horizontal coordinate is given by:

$$X_{soccerPlayer} = \frac{3Xc}{2} - \frac{\left(\frac{Xc}{2} \cdot \frac{YA}{XA}\right)\left(Xf - \frac{Xc}{2}\right)}{\left(\frac{Xc}{2} \cdot \frac{YA}{XA}\right) - Yf}$$

This adjustment ensures that the computed coordinate correctly corresponds to the full field instead of only half of the playground.

2.4.3 Determination of the Breadth Coordinate

In addition to the horizontal coordinate, the depth position of the object must also be determined. This is achieved by projecting the object onto a reference line parallel to the field boundaries and analyzing its relative position.

From the geometric relationship:

$$\sin(C) = \sin(D) = \frac{YA}{AD}$$

Let Q_{Yfr} represent the projected breadth value obtained from the image. Then:

$$Q_{Yfr} = C_{Yfr} \cdot \sin(C)$$

Thus, the real breadth coordinate can be computed as:

$$C_{Yfr} = \frac{Q_{Yfr}}{\sin(C)}$$

In summary, the transformation from image coordinates to real-world coordinates is achieved through a combination of geometric projection and trigonometric relationships. The final position of the object on the football field is expressed as:

$$(X_{soccerPlayer},\ Y_{soccerPlayer})$$

These coordinates provide an accurate representation of the player's location and are essential for subsequent analysis tasks such as offside detection.

2.5. Detection of Lines on the Soccer Pitch

2.5.1. Field Characteristics and Color Space Selection

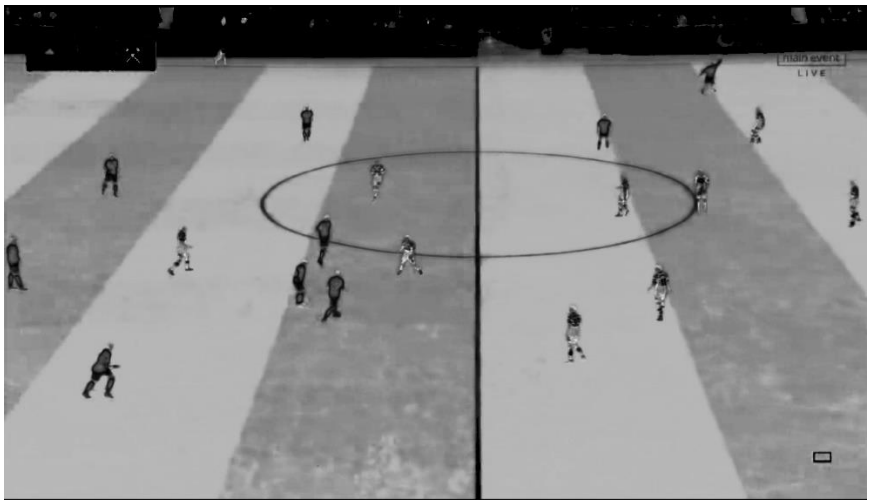


Figure 2.8: Saturation Channel of HSV space

A soccer pitch typically contains alternating light and dark green grass stripes, created by mowing patterns. These stripes form a set of parallel lines along the width of the field and provide an important visual cue for detecting the geometric structure of the pitch. To effectively exploit this characteristic, multiple color spaces including BGR, HSV, and CIE-LAB were evaluated. Experimental results indicate that the Saturation (S) channel in the HSV color space provides the highest contrast between light and dark green regions. This enhanced contrast makes the boundaries between adjacent stripes more distinguishable, which is beneficial for subsequent edge detection.

2.5.2. Edge Detection on the Soccer Field

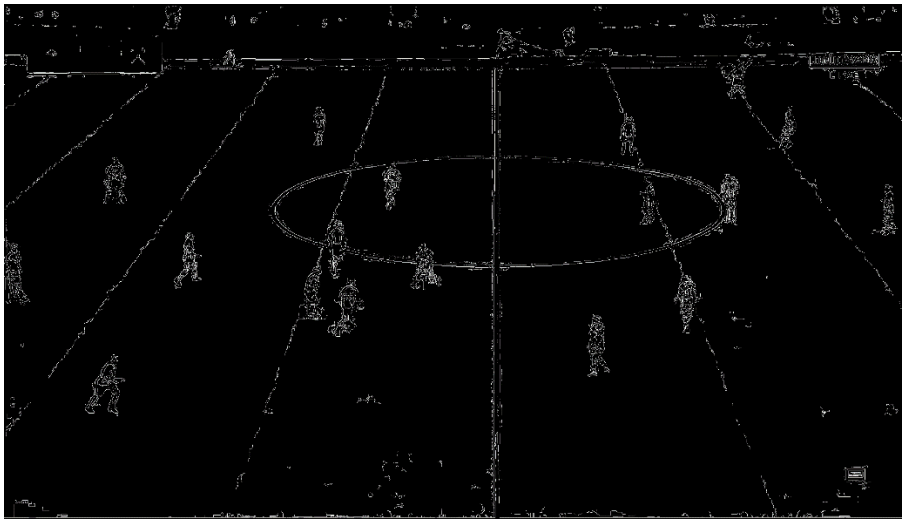


Figure 2.9: Edge detection result using Canny detector.

After extracting the Saturation channel, edge detection is performed to identify the boundaries between the grass stripes. In this work, the Canny Edge Detector is employed due to its robustness and effectiveness in detecting strong edges while suppressing noise.

The Canny algorithm consists of the following steps:

- Noise reduction using a Gaussian filter
- Gradient computation using Sobel operators in both horizontal and vertical directions
- Non-maximum suppression to thin the edges
- Double thresholding to retain significant edges

As a result, the detected edges clearly correspond to the transitions between light and dark grass regions on the field.

2.5.3. Line Detection and Filtering using Hough Transform

Based on the detected edges, the Hough Transform is applied to extract straight lines from the image. This method maps edge points from the image space into the parameter space (ρ,θ), enabling the detection of line candidates. Key parameters such as ρ, θ, and the detection threshold are carefully tuned to achieve accurate results. After detecting multiple line candidates, a filtering step is applied based on the orientation of the lines. Specifically, only lines with angles satisfying:

$$18^\circ < \theta < 89^\circ$$

are retained. This constraint ensures that only lines aligned with the horizontal structure of the pitch are selected, while eliminating irrelevant or noisy detections. The value of 90∘ is avoided due to instability in slope computation. The final output demonstrates that the major parallel lines on the soccer pitch are successfully detected, providing a reliable foundation for constructing the offside line in subsequent stages.

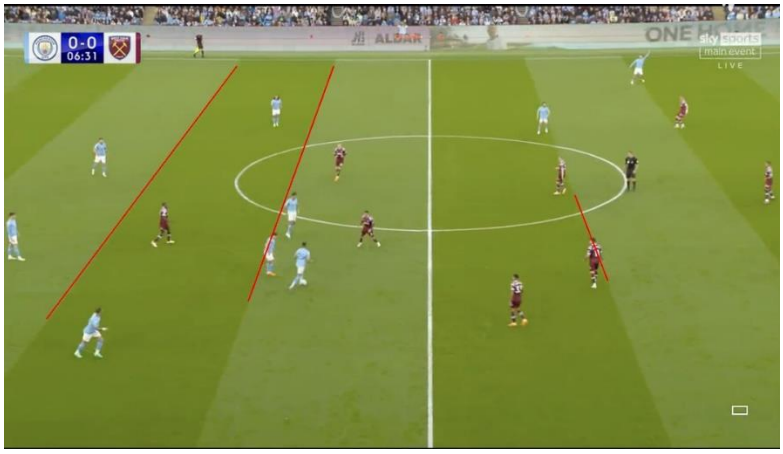


Figure 2.10: Final detected lines on the soccer pitch.

2.6. Vanishing Point Estimation

2.6.1. Perspective Geometry and Vanishing Point

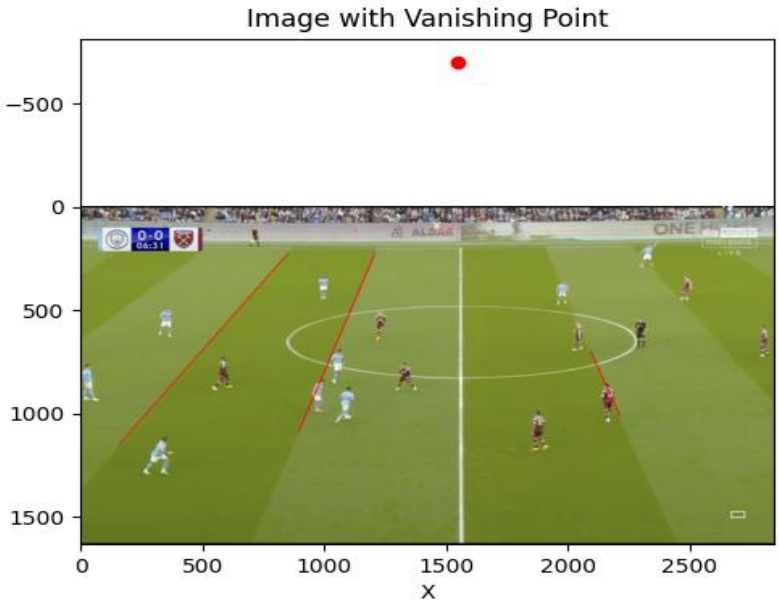


Figure 2.11: Vanishing point when the camera is centered on the pitch

From the detected lines on the soccer field, it can be observed that these lines are not parallel in the 2D image plane due to perspective projection. However, in real-world

3D space, these lines are inherently parallel. This effect occurs because a camera projects a three-dimensional scene onto a two-dimensional image. The vanishing point is defined as the point where parallel lines in 3D space appear to converge in the image. This point provides important information about the scene's geometry and camera perspective. In structured environments such as roads or soccer fields, the vanishing point is typically formed by sets of parallel lines like field markings or grass patterns.

2.6.2. Vanishing Point Estimation using Line Intersections and RANSAC

To estimate the vanishing point, all detected lines are analyzed. For each pair of lines, the intersection point is computed using their line equations. These intersections represent candidate locations for the vanishing point. However, due to noise and incorrect detections, many intersection points are outliers. To obtain a robust estimate, the RANSAC (Random Sample Consensus) algorithm is applied. This method is widely used in computer vision to handle noisy data.

In each iteration:

- Two random intersection points are selected to form a line model
- The distance from all other points to this model is calculated
- Points within a predefined threshold are considered inliers

The algorithm repeats this process multiple times and selects the model that maximizes the number of inliers. Early stopping is applied when a inlier is found, improving efficiency.

2.6.3. Final Vanishing Point Determination and Application

After identifying the best set of inliers, the vanishing point is computed as the average of these points. This reduces noise and provides a stable and accurate estimation. The resulting vanishing point reflects the true perspective direction of the soccer field and plays a crucial role in subsequent processing steps. In particularly, it is used to align the detected field lines and support the construction of the offside line, ensuring that the system correctly accounts for camera perspective.

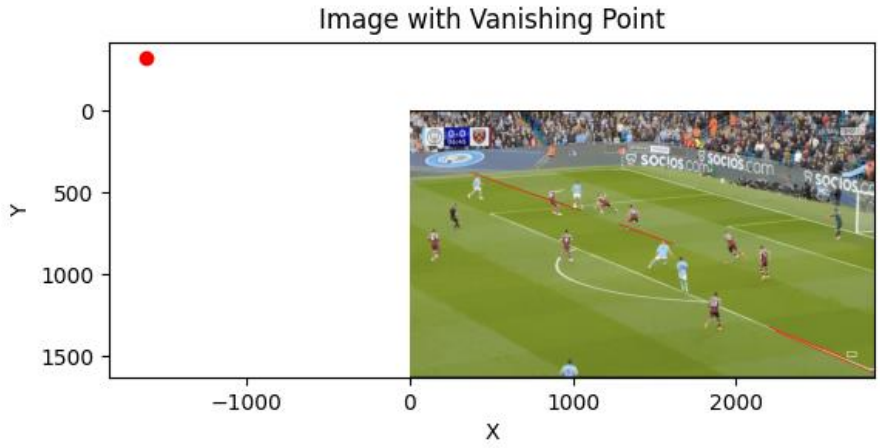


Figure 2.12: Vanishing point when the camera is viewing one end of the pitch

2.7. Keypoint Estimation and Offside Line Construction

2.7.1. Importance of Keypoint Estimation



Figure 2.13: Detected keypoints on players

In offside detection, accurately determining the position of a player is essential. According to the rules of soccer, offside decisions are based on specific body parts such as the shoulders, knees, and feet rather than the entire body region. To achieve this, keypoint estimation is applied to detect and localize important joints of each player. Compared to bounding box-based approaches, keypoints provide a more precise representation of player positions, enabling more accurate geometric analysis. A pre-trained deep learning model, Keypoint R-CNN (ResNet-50-FPN), is used for this task. The input image is first converted to RGB format and normalized before being passed through the model. The model outputs bounding boxes, class labels, confidence scores, and keypoints for each detected person. Detections are filtered based on confidence thresholds, and only reliable keypoints are retained for further processing.

2.7.2. Virtual Line Generation from Vanishing Point

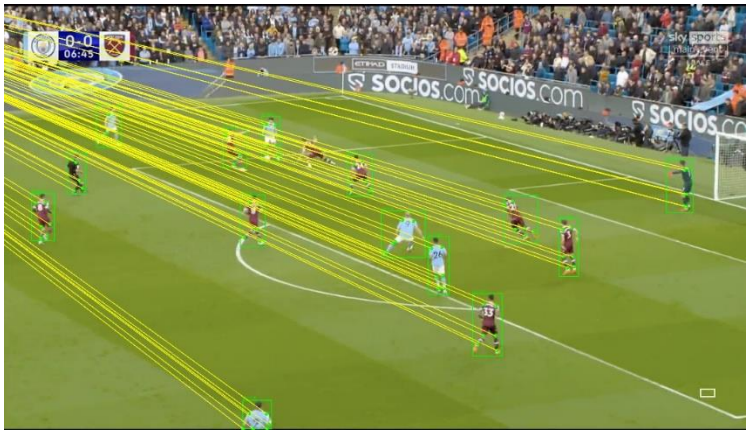


Figure 2.14: Lines drawn from all detected keypoints

After obtaining the vanishing point (x1,y1) from Section 2.6 and the set of detected keypoints (x2,y2), virtual lines are constructed by connecting the vanishing point to each keypoint. Each line represents a projection of a real-world parallel line under perspective transformation. Although these lines appear to converge in the 2D image, they are parallel in 3D space. This step generates a set of candidate lines that may represent potential offside lines.

2.7.3. Offside Line Selection

From the set of generated lines, the goal is to determine the line corresponding to the last defender. This is typically the rightmost valid line among all detected players, excluding the goalkeeper in standard situations. To achieve this, the slopes of all candidate lines are computed. The line with the minimum absolute slope is selected as the final offside line. This criterion ensures that the selected line is consistent with the field perspective and aligns with the defensive line.



Figure 2.15: Final offside line corresponding to the la st defender

CHAPTER 3: IMPLEMENTATION AND SYSTEM DESIGN

3.1 Development Environment and Tools



Figure 3.1:Development environment using PyCharm IDE

In this project, the offside detection system is developed and implemented using the Python programming language within the integrated development environment (IDE) PyCharm. This IDE is widely used due to its powerful features for coding, debugging, and project management, along with a user-friendly interface. The Python language is chosen because of its simplicity, readability, and strong support for computer vision and artificial intelligence applications. Additionally, Python offers a rich ecosystem of libraries that significantly reduce development time and improve implementation efficiency.

Several essential libraries are utilized in this project, including:

- OpenCV: Used for image and video processing tasks such as reading video streams, frame manipulation, and displaying results.
- NumPy: Provides efficient numerical operations and matrix computations, which are fundamental in image processing tasks.
- YOLOv8 (via Ultralytics): Applied for detecting players in video frames with high accuracy and real-time performance.
- Scikit-learn: Used to implement the K-Means clustering algorithm for team classification based on jersey colors.
- FilterPy: Utilized to implement the Kalman Filter for smoothing the offside line, reducing noise, and improving system stability.

Furthermore, the system is executed on a personal computer running the Windows operating system, ensuring sufficiency computational capability for video processing and real-time visualization. Overall, the combination of Python, PyCharm, and

specialized libraries provides a flexible and efficient development environment suitable for building an image-processing-based offside detection system.

3.2 Software Libraries and Tools

3.2.1 Player Detection Using YOLO



Figure 3.2: Player detection using YOLOv8.

In this project, player detection is performed using the Ultralytics YOLOv8 model. YOLO is a real-time object detection algorithm known for its fast processing speed and good accuracy, making it suitable for analyzing football videos. The model is applied to each video frame to detect objects and return bounding boxes with class labels. Only objects classified as “person” are retained as players, while small detections are filtered out to reduce noise. The resulting bounding boxes provide the positions of players in each frame and are used as input for subsequent steps such as team classification, last defender estimation, and offside detection.

3.2.2 Team Classification Using K-Means

After detecting players, the system classifies them into two teams using the K-Means clustering algorithm. This method groups players based on the dominant color extracted from their bounding boxes, which typically corresponds to the jersey color. The average color of each player region is computed and used as input for clustering. K-Means is applied with K=2 to separate players into two groups representing two teams. The clustering results are then used to distinguish attackers and defenders, which is essential for determining the attacking direction and evaluating offside situations in later stages.

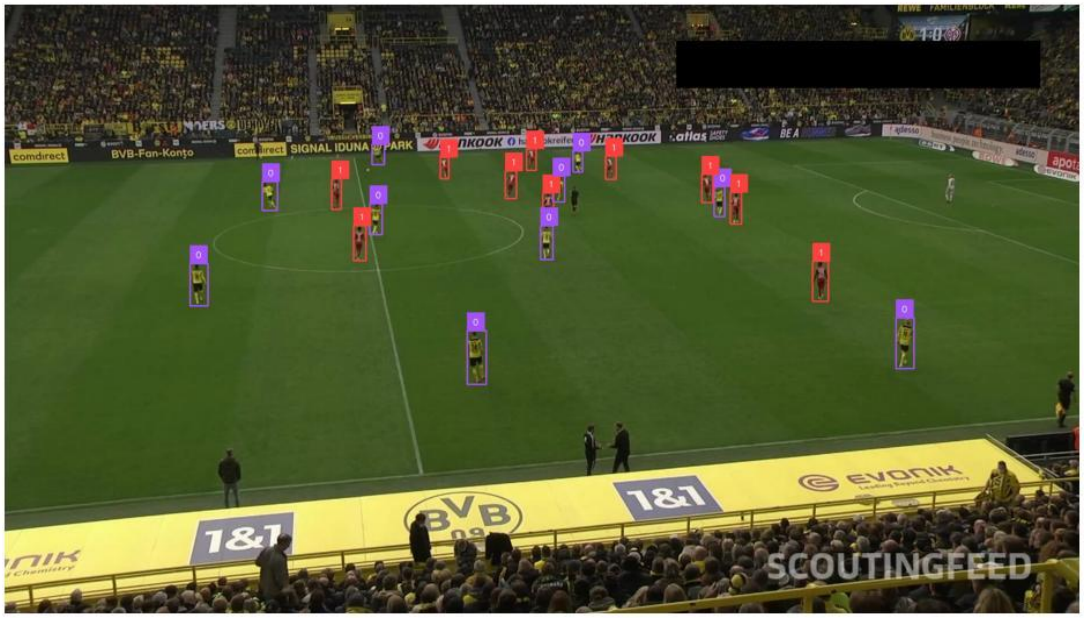


Figure 3.3: Team classification using K-Means clustering.

3.2.3 Offside Line Estimation Using Kalman Filter

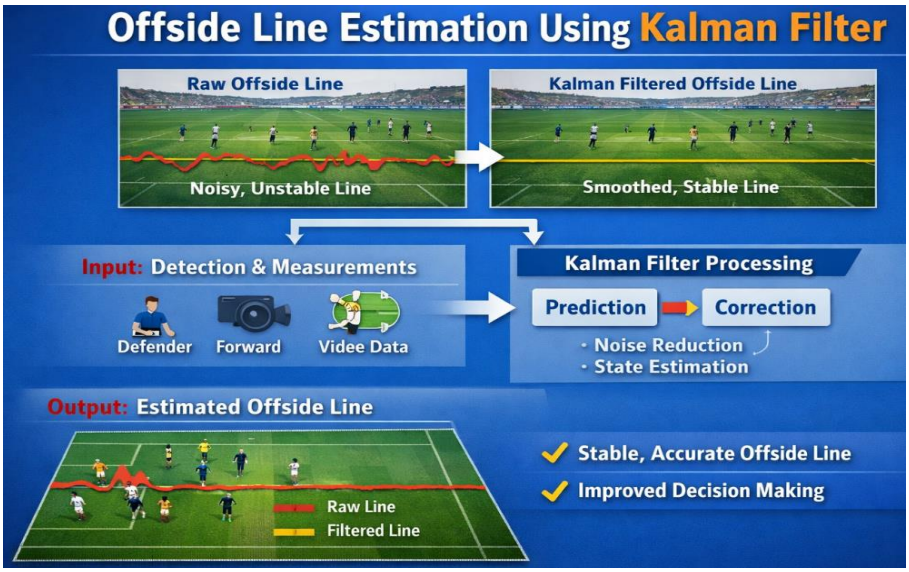


Figure 3.4: Smoothed offside line estimation using Kalman Filter

To ensure a stable and consistent offside line, the system applies a Kalman Filter to smooth the position of the last defender across video frames. Since player detection may fluctuate due to noise or occlusion, the raw position of the last defender can vary significantly between frames, leading to an unstable offside line. In this implementation, the Kalman Filter predicts and updates the horizontal position of the offside line based on previous states. Additionally, a short history of recent positions is maintained to further smooth the output. This combination helps reduce sudden

changes and produces a more stable offside line, improving the visual consistency and reliability of the offside detection system.

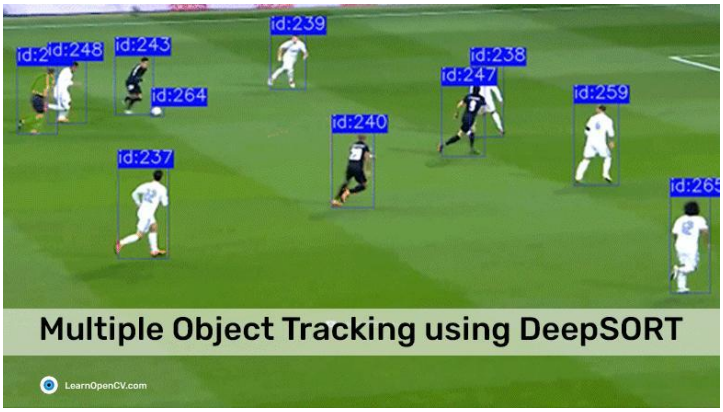
3.2.4 OpenCV



Figure 3.5: OpenCV image processing library

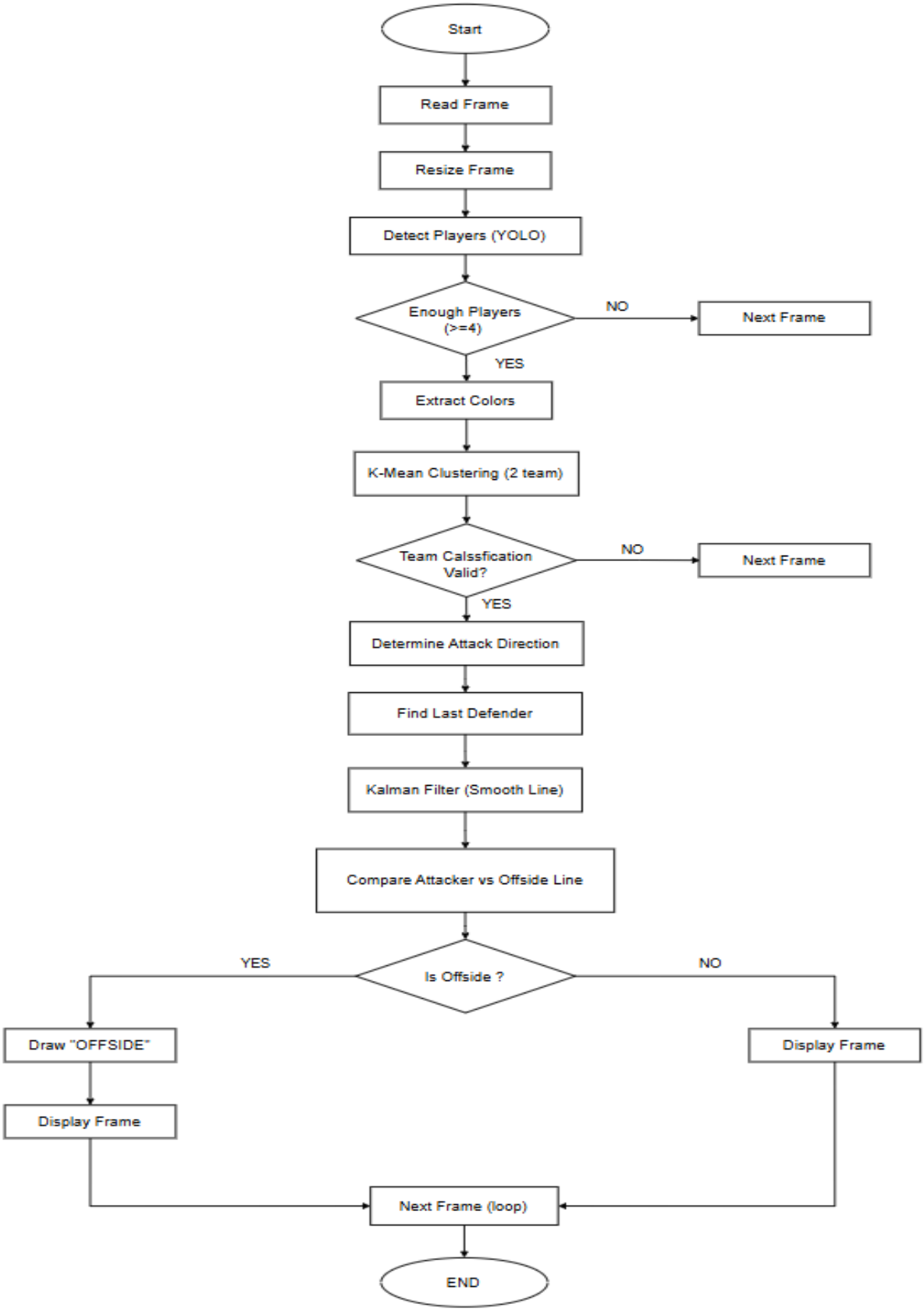
In this project, OpenCV is utilized as the primary library for handling image and video processing tasks. OpenCV provides a wide range of functions for reading video frames, detecting objects, applying transformations, and visualizing results in real time. Within the system, OpenCV is responsible for capturing video frames from the match footage and preprocessing them for subsequent detection steps. Functions such as color conversion, thresholding, contour detection, and line drawing are applied to highlight player positions, field lines, and the offside line. By integrating OpenCV with YOLO for player detection and Kalman Filter for line stabilization, the framework achieves a smooth and accurate offside detection workflow. This implementation demonstrates how OpenCV acts as the backbone for real-time image processing, bridging raw video input with advanced analytical algorithms to enhance the reliability of offside decision-making.

3.2.5 DeepSORT for Player Tracking

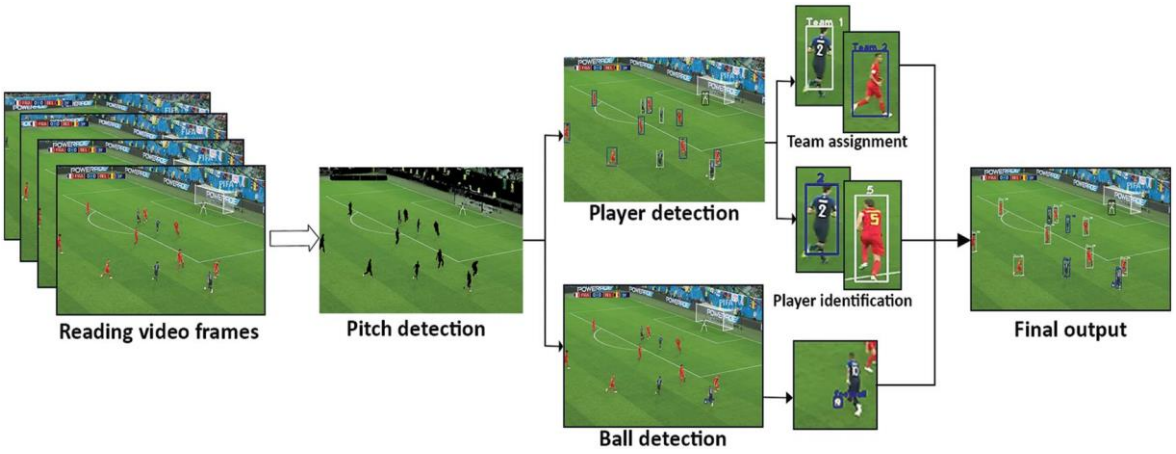


DeepSORT (Simple Online and Realtime Tracking with a Deep Association Metric) is used to track multiple players across video frames. Instead of detecting players independently in each frame, DeepSORT assigns a unique ID to each detected object and maintains its identity over time. The algorithm combines motion prediction using a Kalman Filter with appearance feature extraction, allowing it to match detected players even when they move quickly or are partially occluded. This improves temporal consistency compared to frame-by-frame detection methods.

3.3 Flowchart of the offside detection algorithm.



3.4. System Workflow of the Offside Detection System



3.4.1. Data Acquisition and Pre-processing

The system takes a video file (e.g., football1.mp4) as the main input source. The video is processed frame-by-frame using the OpenCV library. Each frame is resized to a fixed resolution (1480×820) to ensure consistency and maintain real-time processing performance.

Basic pre-processing steps are applied before detection:

- Frame resizing to standard dimensions
- Filtering out small objects based on bounding box height
- Keeping only relevant regions likely to contain players

These steps help reduce noise and improve the accuracy of subsequent detection and tracking stages.

3.4.2. Player Detection and Tracking

Player detection is performed using YOLOv8. The model identifies players in each frame and outputs bounding boxes in the format (x1, y1, x2, y2) along with confidence scores. To maintain player identity across frames, the system integrates DeepSORT. This tracking algorithm assigns a unique ID to each detected player and ensures temporal consistency.

Key advantages of using DeepSORT in this system:

- Maintains stable identities for players across frames
- Reduces flickering and misidentification
- Enables reliable offside decision based on consistent tracking

Additionally, a Kalman Filter is used internally to predict motion and smooth tracking results, especially during fast movements or partial occlusions.

3.4.3. Team Classification and Attack Direction Estimation

To distinguish between two teams, the system uses KMeans clustering based on jersey color features.

The process includes:

- Extracting the average color from each player's bounding box
- Applying KMeans clustering with k = 2
- Assigning players into two groups corresponding to two teams

After team classification, the system estimates the attacking direction by computing the average horizontal position (X-axis) of both teams:

- If the attacking team has a larger average X value → attacking from left to right
- Otherwise → attacking from right to left

This step is essential for correctly interpreting spatial relationships in offside detection.

3.4.4. Last Defender Identification and Offside Line Estimation

From the set of defenders, the system determines the last defender based on spatial positioning.

Steps involved:

- Compute the effective body edge of each defender using a weighted combination of bounding box coordinates
- Sort defenders along the X-axis
- Select the second-last defender (according to offside rules)

The offside line is then drawn at this position.

To improve stability, the position of the offside line is smoothed using a Kalman Filter combined with a short-term averaging buffer. This reduces jitter caused by detection noise or camera movement and produces a visually stable line.

3.4.5 Offside Player Determination

The system evaluates each attacking player to determine whether they are in an offside position.

For each attacker:

- Compute the forward-most edge relative to the attacking direction
- Compare it with the offside line position

- Calculate the positional difference (distance threshold)

A player is considered a potential offside candidate if:

- They are beyond the last defender
- The positional difference exceeds a predefined threshold
- They are located on a valid attacking zone

Among all candidates, the system selects the player with the largest violation distance.

To prevent unstable results, a temporal consistency mechanism is applied:

- The detected offside player ID is maintained for several frames (hold counter)
- The system only updates the offside player when a significant change occurs

This ensures that the offside decision does not fluctuate rapidly between players.

3.5. Experimental Results and Evaluation

3.5.1. Experimental Setup



The proposed system is evaluated using a football video sequence (*football1.mp4*) as input. The video is processed frame-by-frame using the developed pipeline, which includes player detection, team classification, offside line estimation, and decision logic. This evaluation focuses on real-world scenarios where multiple players are moving dynamically, which helps assess the robustness and stability of the system.

3.5.1. Team Classification Analysis (ATK vs DEF)

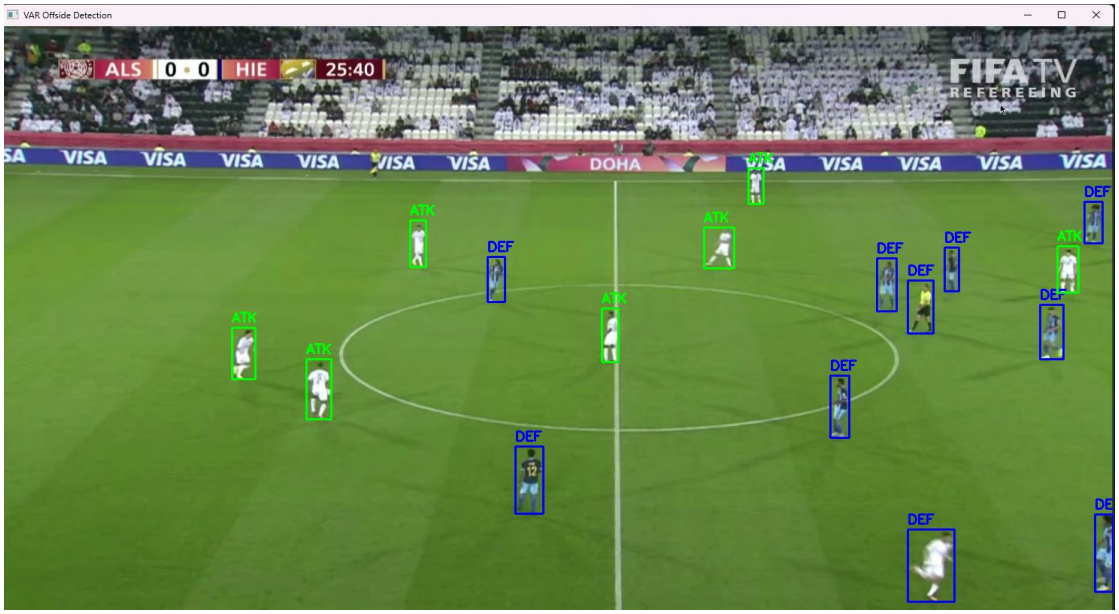


Figure 3.6: Player Detection and Team Classification Result

The system separates players into two teams by analyzing the dominant color features extracted from each detected bounding box. This classification serves as the foundation for determining attacking and defending sides.

The computation process is described as follows:

- Color feature extraction:

For each detected player region, the image is resized and the average color is computed:

$$C = \frac{1}{N}\sum_{i=1}^{N}(R_i, G_i, B_i)$$

where:

N is the number of pixels in the cropped region. This produces a representative RGB vector for each player.

- Clustering using K-Means:

All color vectors are grouped into two clusters using K-Means:

$$\arg\min \sum_{k=1}^{2}\sum_{x \in S_k} \|x - \mu_k\|^2$$

where μk represents the centroid of each team cluster

- Cluster consistency across frames:

To maintain stable labeling, the cluster centers are updated using a smoothing strategy:

$$C_{new} = 0.8 \cdot C_{prev} + 0.2 \cdot C_{current}$$

Additionally, Euclidean distance is used to match new clusters with previous ones:

$$d = \|C_{current} - C_{reference}\|$$

This prevents label switching between frames.

- Attacking direction estimation:

The system estimates the attacking direction by comparing the average horizontal positions:

$$\bar{x}_{atk} = \frac{1}{n}\sum x_{atk}, \quad \bar{x}_{def} = \frac{1}{m}\sum x_{def}$$

If xatk > xdef, the attacking direction is from left to right, and vice versa.

3.5.2. Offside Visualization Result

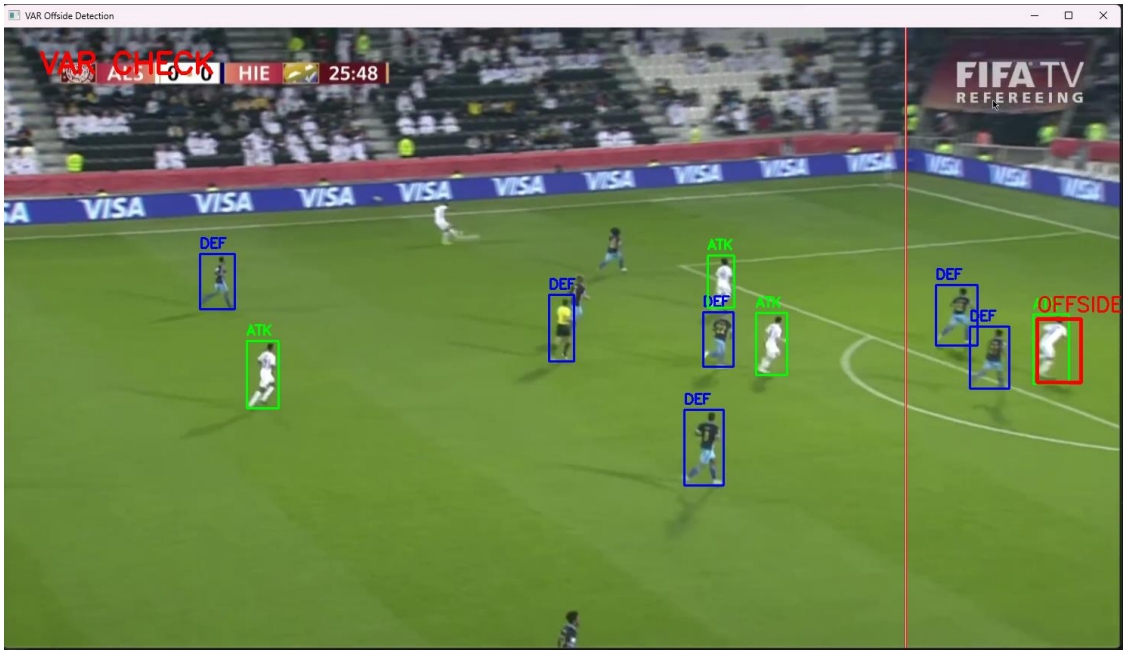


Figure 3. 7: Offside detection visualization with labeled players and offside line.

The system provides a visual representation of the offside situation by overlaying detection results directly onto the video frame. This visualization allows users to clearly observe player roles and spatial relationships.

The visualization process includes:

+ Player role display:

Each detected player is labeled using bounding boxes:

- Green boxes indicate attacking players (ATK)
- Blue boxes indicate defending players (DEF)

+ Offside line rendering:

A vertical line is drawn at the position of the last defender:

$$(x_{line}, 0) \rightarrow (x_{line}, H)$$

This line serves as the visual boundary for offside evaluation.

Offside indication:

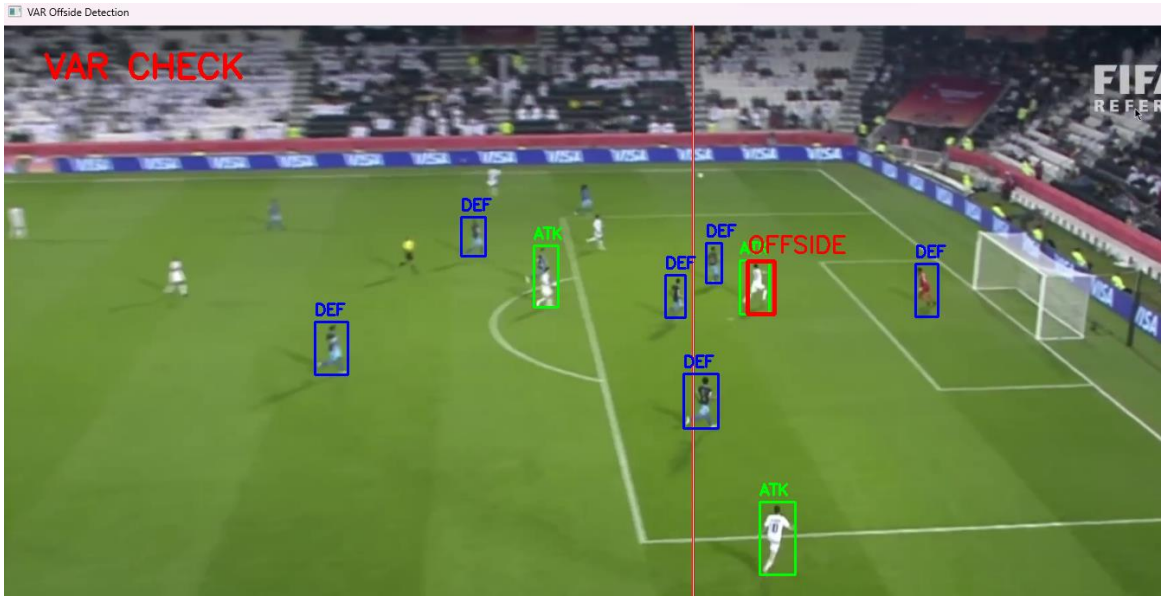
When an offside situation occurs:

- The violating player is highlighted with a red bounding box
- The label “OFFSIDE” is displayed above the player

System feedback:

The text “VAR CHECK” appears on the screen to indicate that an offside event has been detected.

3.5.3. Offside Detection under Perspective Variation



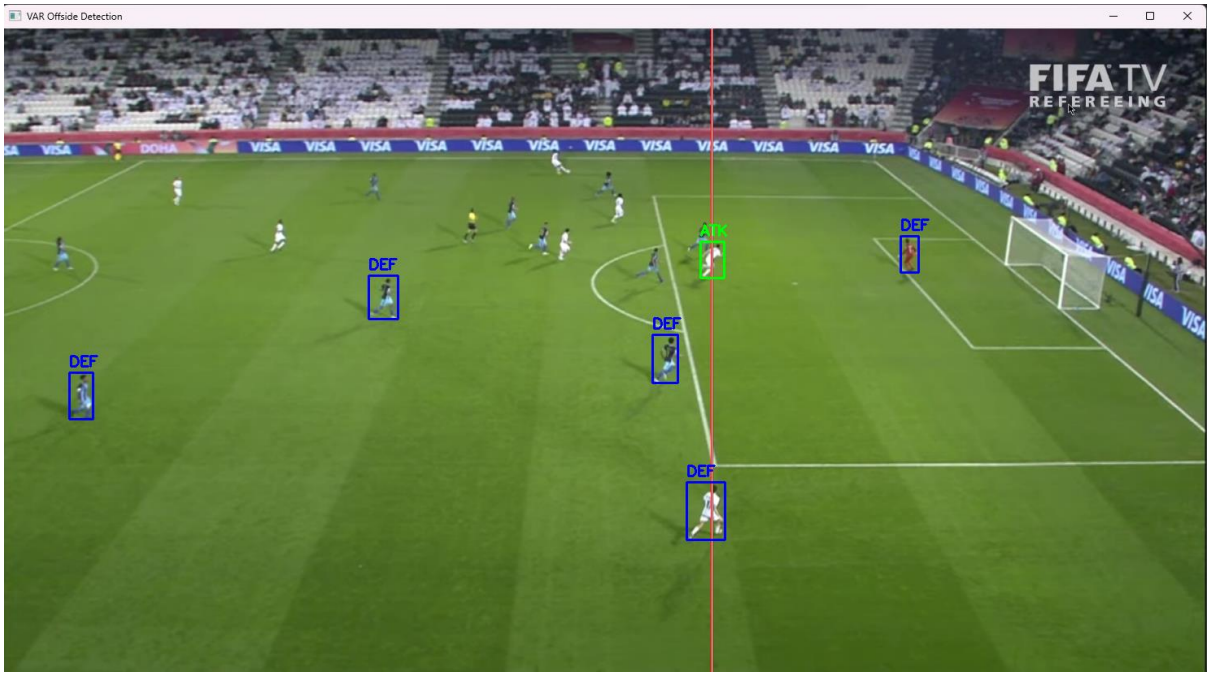


Figure 3.8: Offside detection under different camera perspectives.

The system maintains reliable offside detection even when the camera angle changes, as shown in Figure 3.8. Despite variations in perspective and player distribution, the algorithm consistently identifies the last defender and evaluates attacking players relative to the offside line. The computation process remains stable due to the use of normalized spatial features and directional analysis

+ Perspective-invariant player representation:

Each player is still represented by a bounding box: (x_1, y_1; x_2, y_2)

Instead of using the full box, the system focuses on a weighted horizontal position:

$$x_{edge} = \alpha x_1 + (1 - \alpha) x_2$$

This approximation reduces the impact of camera distortion by emphasizing the leading edge of the player.

+ Relative position consistency:

Even under different camera angles, offside is determined using relative horizontal ordering:

$$d = x_{atk} - x_{def}$$

Since all players are evaluated in the same coordinate system, the relative difference remains valid regardless of perspective.

+ Dynamic offside line estimation:

The last defender is computed from sorted defender positions:

$$X = \{x_1, x_2, ..., x_n\}$$

After sorting:

- Attacking right → xline =x_1
- Attacking left → xline=x_2

This ensures the offside line adapts correctly to player distribution in any view.

+ Temporal smoothing for stability:

To handle camera motion and frame variation, the offside line is stabilized using:

$$xk = Fxk - 1 + w, zk = Hxk + v$$

Combined with: $x_{final} = 0.7\ median + 0.3\ mean$

This reduces noise caused by perspective shifts and detection fluctuations.

+ Offside condition under varying view:

An attacking player is flagged when: $d > \text{thresold}$

The most advanced player is selected using: $\max(d)$ and highlighted as the offside player.